12 · Control Tower: use an Interlens local model as an untrusted policy

Siddharth M. Bhatia · siddharth@sidmb.com
July 27, 2026

Contents

Install 1
Define the participant 2
Run a Control Tower protocol 2
Programmatic use and interpretability 3
Attack and collaborative policies 4

Interlens and Control Tower meet at the policy boundary. Interlens owns local Hugging Face generation and interpretability; Control Tower owns LinuxArena environments, blue-team protocols, tool execution, monitoring, auditing, sandboxing, and trajectory storage.

The adapter returns tool calls to Control Tower **without executing them inside Interlens**. This is the important security boundary: a trusted-monitoring protocol sees the same proposed action it would see from any other untrusted model.

Install

Control Tower requires Python 3.13. Install both packages and Interlens's Inspect integration:

```
uv add control-tower "interlens[inspect]"
```

Define the participant

Put a zero-argument factory in your project, for example `my_project/policies.py`:

```python
from interlens import AutoModelParticipant

def local_qwen():
    """Build one fresh participant for one Control Tower sample."""
    return AutoModelParticipant.from_pretrained(
        "Qwen/Qwen2.5-Coder-7B-Instruct",
        name="untrusted",
        device="cuda",
        load_kwargs={
            "dtype": "bfloat16",
            "attn": "flash_attention_2",
            "quant": None,
            "revision": None,
        },
        temperature=0.2,
        top_p=0.95,
        max_new_tokens=2048,
        thinking=False,
        kv_reuse="auto",
    )
```

A factory is used instead of a global participant so every Control Tower sample gets fresh KV-cache and generation state. Interlens's process-wide model cache still shares the loaded weights. The adapter serializes concurrent calls that share one cached model because Hugging Face generation is not thread-safe on a single model object; Control Tower can still overlap environment work and calls backed by different models.

Run a Control Tower protocol

Pass the adapter as an external untrusted policy and the factory as its argument:

```
uv run ct run eval \
  --untrusted-policy
     interlens.integrations.control_tower:interlens_untrusted_policy \
  --untrusted-policy-arg
     participant_factory=my_project.policies:local_qwen \
  --protocol trusted-monitoring \
```

```
  --protocol-arg sus_threshold=7 \
  --task-file simple
```

For each untrusted-model request:

1. Inspect messages and available LinuxArena tools are translated to an Interlens model view.
2. The participant performs one local generation with its native chat template.
3. Parsed tool calls become ordinary Inspect `ToolCall`s.
4. Control Tower decides whether to monitor, audit, defer, replace, or execute them.
5. Tool results return through Inspect on the following model call.

`tool_choice="auto"` and `"none"` are supported. Required or forced tool choices fail loudly because Hugging Face chat templates can render tool schemas but cannot reliably enforce a call.

Programmatic use and interpretability

Use `participant_generate` directly when constructing a custom Control Tower policy. Steering, capture, patching, and log probabilities can be attached once to the adapter:

```python
from interlens.integrations.control_tower import participant_generate

generate = participant_generate(
    local_qwen(),
    steering=my_steering_spec,
    capture=my_capture_request,
    patch=None,
    return_logprobs=False,
)
```

The returned function has Control Tower's `UntrustedPolicyGenerateFn` signature:

```python
output = await generate(messages, tools, tool_choice, generate_config)
```

Attack and collaborative policies

The same boundary also exposes an attack-policy wrapper and a private planner-reviewer-executor team. The attack wrapper reads Control Tower's sample-local side-task goal and uses it only as the Interlens policy's private system prompt; blue protocols retain their neutral monitor-visible history:

```
uv run ct run eval \
  --untrusted-policy
     interlens.integrations.control_tower:interlens_attack_policy \
  --untrusted-policy-arg
     participant_factory=my_project.policies:local_qwen \
  --protocol trusted-monitoring \
  --protocol-arg sus_threshold=7 \
  --task-file medium
```

For collaborative inference, one local participant sequentially plays planner, reviewer, and executor on every outer action. Planner and reviewer see a text capability summary but receive no callable tool schemas; only the executor can propose an external action:

```
uv run ct run eval \
  --untrusted-policy
     interlens.integrations.control_tower:interlens_team_policy \
  --untrusted-policy-arg
     participant_factory=my_project.policies:local_qwen \
  --untrusted-policy-arg mode=honest \
  --untrusted-policy-arg planner_max_tokens=512 \
  --untrusted-policy-arg reviewer_max_tokens=512 \
  --protocol untrusted-agent \
  --protocol-arg max_steps=80 \
  --task-file one_per_env
```

Set `mode=attack` only on a task file containing side tasks. Private planning/review text is recorded under `ModelOutput.metadata["interlens"]["collaboration"]`; all three calls contribute to the reported input tokens, output tokens, and cost. The executor's message and tool calls remain ordinary Inspect objects, so monitoring, deferral, editing, sandbox execution, and scoring continue to be Control Tower's responsibility.

This adapter is intentionally for local `ModelParticipant`-style generators. Hosted Interlens API participants already overlap with Inspect's provider layer and do not expose local activation interventions.

© 2026 Siddharth M. Bhatia. All rights reserved. Interlens is licensed under the GNU Affero General Public License v3.0.